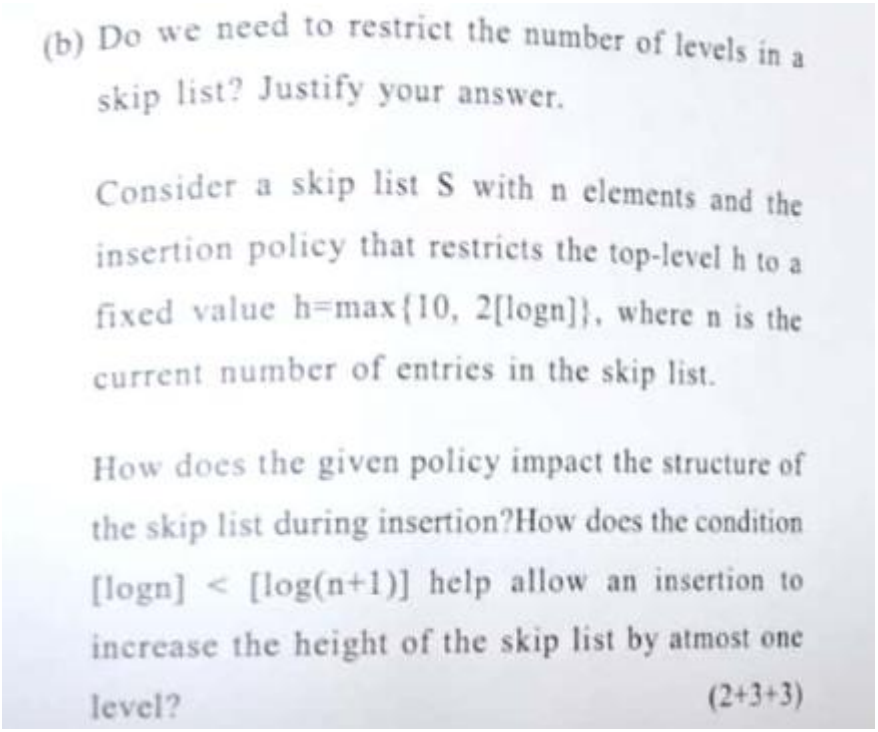
Skip List

https://eduarmandov.wordpress.com/wp-content/uploads/2017/05/c_c-data-structures-and-algorithms-in-c.pdf

9.4.1



(b) Do we need to restrict the number of levels in a skip list? Justify your answer.

Consider a skip list S with n elements and the insertion policy that restricts the top-level h to a fixed value h=max{10, 2[logn]}, where n is the current number of entries in the skip list.

How does the given policy impact the structure of the skip list during insertion?How does the condition [logn] < [log(n+1)] help allow an insertion to increase the height of the skip list by atmost one level? (2+3+3)

✅ Part 1 — Do we need to restrict number of levels?

Yes. Why?

Skip lists assign random heights to nodes.
In theory, a node *could* keep getting "promoted" to higher and higher levels.

Without restriction:

- You could accidentally get **very tall towers** of levels.
- This wastes memory.
- Searching becomes slower because you climb unnecessary empty levels.
- Expected height $O(\log n)$ is no longer guaranteed.

👉 **Therefore, skip lists must cap the maximum height**, normally to something proportional to $\log n$.

✅ How does a node GET those levels? (Promotion)

When you insert a node, you toss a coin repeatedly (in many implementations):

- If it's **heads**, the node gets promoted to the next level.
- If it's **tails**, you stop.

Example:

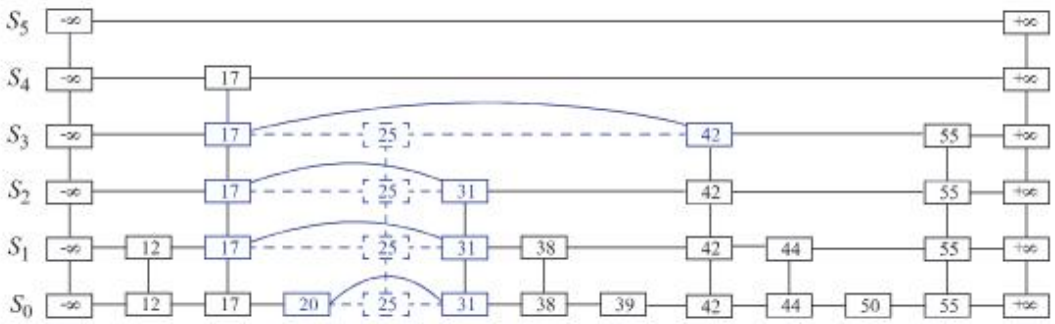
Insert key = 20
Coin tosses: H, H, T
→ First toss = H → level 0 → level 1
→ Second toss = H → level 1 → level 2
→ Third toss = T → stop
Node height = 2

So 20 appears in:

```mathematica
Level 2
Level 1
Level 0
```

This is called promotion.

You *promote* the node each time coin toss = heads.



✅ PART 2 — What does

$$h = \max\{10,\ 2\lfloor \log n \rfloor\}$$

do during insertion?

We use **log base 2** because that's standard.

🟦 Case 1: Small n

n = 1

floor(log 1) = 0
h = max(10, 2×0) = 10

n = 5

floor(log 5) = 2
h = max(10, 4) = 10

n = 50

floor(log 50) = 5
h = max(10, 10) = 10

👉 **Skip list height stays = 10 for all n up to 100.**

So for a long time:

- Insert node
- Randomly promote
- But cap its height at **10**

The structure stays stable and does not grow yet.

🟦 Case 2: Now n becomes large enough to grow height

Watch what happens around **n = 100**.

n = 100

floor(log 100) = 6
h = max(10, 12) = 12

👉 The maximum height **increases from 10 → 12**

What happens during the insertion that made n = 100?

- Skip list adds **one new empty level** at the top (level 10 → level 11 or level 11 → level 12 depending on index)
- All existing nodes remain at their previous heights
- Only future inserts can use the new level

🟦 Case 3: For many insertions, height stays 12

Try n from 100 to 255:

n = 101 → floor(log) = 6 → h = 12
n = 150 → floor(log) = 6 → h = 12
n = 200 → floor(log) = 7 → h = 14 (**but only after crossing 128**)
n = 130 → floor(log) = 7 → h = 14

Let's examine that carefully.

✅ Correct final statement (clean and accurate):

✔ **When ⌊log(n+1)⌋ increases (which happens when n+1 becomes a power of 2),**

the **allowed maximum height** of the skip list increases **by 2** (because h = 2⌊log n⌋).

✔ **BUT**

during that insertion, the skip-list **algorithm** can only **create ONE new physical top level**.

✔ **Therefore**

in that iteration, only one new level becomes available for actual use,
even though the maximum allowed height has increased by more than 1.

🎯 Example: n + 1 = 8 (a power of 2)

This means:

- Before insertion: n = 7
- After insertion: n = 8

And **8 is a power of 2**.

So this is **exactly** the moment where

$$\lfloor \log n \rfloor < \lfloor \log(n+1) \rfloor.$$

Let's go step by step.

⭐ STEP 1 — Compute the floor logs

Before insertion (n = 7):

$$\lfloor \log_2 7 \rfloor = 2$$

After insertion (n+1 = 8):

$$\lfloor \log_2 8 \rfloor = 3$$

So, **the floor(log) value increased by exactly 1**, as expected.

⭐ STEP 2 — Compute the allowed maximum height (h = 2 * floor(log n))

Before insertion:

$$h_{\text{old}} = 2 \cdot 2 = 4$$

After insertion:

$$h_{\text{new}} = 2 \cdot 3 = 6$$

✔ **Mathematically, the allowed height jumps from 4 → 6.**

That is a jump of **2 levels**.

So yes — based on the formula:

"The height wants to increase by 2 levels."

But we're not done.

⭐ STEP 3 — What does the *skip-list insertion algorithm actually do*?

Here is the KEY part:

❗ **During a single insertion, the skip list can only physically add ONE new top level.**

It cannot add two in one shot.

This is because one insertion can only:

- insert one node
- promote that single node upward
- add **one** new top level for that promotion

It does NOT re-promote every old node,
and it does NOT create two levels in one step.

When inserting the 8th element:

1. **⌊log n⌋ jumps from 2 → 3**
2. **Allowed height jumps from 4 → 6**
3. **But the skip-list insertion physically adds only ONE new level**, so actual height becomes 5
4. The next insertion can raise it from 5 → 6
5. After that, actual height = allowed height

Advantages of Skip List:

- The skip list is solid and trustworthy.
- To add a new node to it, it will be inserted extremely quickly.
- Easy to implement compared to the hash table and binary search tree
- The number of nodes in the skip list increases, and the possibility of the worst-case decreases
- Requires only ?(logn) time in the average case for all operations.
- Finding a node in the list is relatively straightforward.

Disadvantages of Skip List:

- It needs a greater amount of memory than the balanced tree.
- Reverse search is not permitted.
- Searching is slower than a linked list
- Skip lists are not cache-friendly because they don't optimize the locality of reference